

thump

A math + CS project plan for an audio sample similarity search engine

WAV -> signal -> Fourier representation -> frequency features -> feature matrix -> SVD -> latent representation -> similarity search

Goal. Given a query drum sample, return the k samples in your library that are most acoustically similar. Start with kicks only so the system learns differences between kicks rather than merely separating kicks from snares or hats.

Phase 0 - Set up the project

Start with roughly 50-100 kick samples that you already own. Keep the code modular so each mathematical step can be tested independently.

```
sample-search/  
  data/kicks/  
  src/audio.py  
  src/fourier.py  
  src/features.py  
  src/similarity.py  
  src/svd.py  
  experiments/  
  main.py  
  requirements.txt  
  README.md
```

Good rule: let libraries handle boring I/O, but do not call a black-box implementation of the mathematical idea you are currently trying to learn before you understand and test your own basic version.

Phase 1 - Understand the raw audio

Load one WAV file and inspect what the computer actually sees. A sampled audio signal is just a long vector of amplitudes.

For one file, inspect: sample rate, number of samples, duration, mono vs. stereo, and amplitude range. Plot amplitude versus time.

Before moving on, be able to explain: what 44.1 kHz means; why an audio recording can be treated as a vector; what each coordinate represents; and why two similar-sounding kicks may still have very different raw sample-by-sample vectors.

Phase 2 - Build the Fourier representation

Treat the kick as a signal $f(t)$. For each frequency direction u_i , compute a projection coefficient $c_i = \langle f, u_i \rangle$. Intuitively, each coefficient measures how strongly that frequency is present in the kick.

For learning purposes, implement a small discrete Fourier transform from the mathematical definition before switching to a fast FFT library. Then compare the outputs so you can see that the library is accelerating the same mathematical operation.

Create a frequency-spectrum plot: horizontal axis = frequency, vertical axis = magnitude. You should be able to compare a boomy kick and a clickier kick visually.

Phase 3 - Turn one kick into a feature vector

Divide the frequency range into a small number of bands - for example, 10 equal-width bands as a deliberately simple baseline. For each band, summarize the spectral energy that falls inside it.

One kick now becomes a vector x in $\mathbb{R}^{10}$, where each coordinate represents energy in one frequency range. Verify this manually using a boomy kick, a balanced kick, and a bright/clicky kick.

Do not worry yet about whether equal-width bands are optimal. The baseline is useful precisely because you can later measure how better frequency scales improve the system.

Phase 4 - Build search without SVD

Encode every kick as a feature vector and stack those vectors as rows of a matrix A . If you have m kicks and n features, A is an $m \times n$ matrix.

Implement Euclidean distance $d(x,y) = \|x-y\|_2$. Given a query kick, compare its feature vector to every indexed kick, sort the distances, and return the closest matches.

At this point, the project already works as a basic search engine.

Phase 5 - Compare similarity metrics

Add cosine similarity: $\text{sim}(x,y) = (x^T y) / (\|x\| \|y\|)$. Listen to the top matches from Euclidean distance and cosine similarity.

Your first real experimental question is: **Which mathematical definition of similarity better matches your own perception of similar kicks?**

Phase 6 - Add SVD

Once the baseline search works, compute $A = U \Sigma V^T$. Plot the singular values in descending order. The singular vectors give new orthogonal directions that capture the strongest structure in the kick library.

Measure how much squared singular-value energy is retained by the first k components: $\sum_{i=1..k} \sigma_i^2$ divided by $\sum_{i=1..n} \sigma_i^2$.

Choose a smaller k and project the kicks into that reduced space.

Phase 7 - Create the sound space

Each kick now gets a latent vector z in $\mathbb{R}^k$ instead of its original frequency-feature vector. Run nearest-neighbor search in this reduced SVD space and compare the results to the raw-feature search.

Conceptually: original frequency features -> SVD latent sound space -> distance or cosine similarity -> nearest kicks.

Phase 8 - Visualize the sound space

Plot each kick using its first two or three SVD coordinates. Each point is one sample. Listen to neighboring clusters and see whether regions correspond to recognizable qualities such as boomy, short, distorted, acoustic, or clicky.

The interesting part is that you did not manually define those labels. You are interpreting the structure that emerged from the dataset.

Phase 9 - Run an actual experiment

Choose around 10 query kicks and retrieve the top matches under four systems: raw features + Euclidean, raw features + cosine, SVD features + Euclidean, and SVD features + cosine.

Blind-listen and rate the returned samples for perceived similarity. This lets you investigate whether dimensionality reduction actually improves retrieval quality. A negative result is still a valid and interesting result.

Phase 10 - Break your intentionally simple assumptions

Once the baseline is working, question it. Equal-width frequency bands may not align well with human hearing, and frequency content alone may miss important differences in duration, transient shape, decay, pitch envelope, or dynamics.

Add richer features one at a time and measure whether they improve perceived similarity. This keeps the project scientific instead of turning it into feature soup.

Phase 11 - Scale the dimensionality

Try 10, 25, 50, 100, or even hundreds of frequency features. As the original representation grows, SVD becomes more meaningful because it can compress a large feature space into a much smaller one while preserving much of the variation.

Track the tradeoff among dimensionality, retained singular-value energy, retrieval quality, and search speed.

Phase 12 - Make it usable

Only after the algorithm works, build a simple interface. Let the user select a query sample, audition it, and play the top returned matches with similarity scores.

A later version could index an entire sample folder once and let a producer drag in any sound to retrieve the nearest samples instantly.

Recommended learning order

#	Milestone
1	Load WAV
2	Understand the waveform
3	Implement simple Fourier analysis
4	Visualize the frequency spectrum
5	Create frequency-band vectors
6	Encode the kick library
7	Euclidean search
8	Cosine search
9	SVD
10	Reduced sound space
11	Compare methods
12	Improve the audio representation
13	Build the UI

Core principle: build the simple mathematical version yourself first. Let the weaknesses you discover motivate the next technique. That way Fourier analysis, similarity metrics, and SVD become tools that solve problems you have personally encountered, rather than boxes you checked for a portfolio.